WikiArt Dataset Analysis

Structural analysis & data-quality audit of a 27-style, 81,444-image painting corpus

1. Executive summary

This report documents a structural analysis and data-quality audit of a WikiArt painting corpus of **81,444** images organised into **27** style folders, accompanied by two label files, `classes.csv` and `wclasses.csv`. All findings were established by directly inspecting the data; none are inferred from folder names alone. The dataset is structurally clean — **every one of the 81,444 images opens and decodes as a valid RGB image (0 corrupt)** — but the two label files are mutually inconsistent in their schema and, critically, use two different broken text encodings for accented artist names that defeat naive joins.

2. Source-of-truth hierarchy

A single, explicit hierarchy governs the analysis:

1. Filesystem (27 folders)	Authoritative image inventory and style label. The only artifact that exactly represents the images on disk; the folder name equals the primary `classes.csv` style for **every** overlapping row (0 mismatches, verified).
2. classes.csv	Primary human-readable metadata (artist name, dimensions, phash, train/test split, style). Joined onto the inventory as an enrichment.
3. wclasses.csv	Validation / reconciliation only. Numeric-ID encoding with no legend file present; artist and genre IDs are left unresolved. Inconsistencies are reported, never silently discarded or overwritten.

3. Dataset overview

Images on disk	81,444 (all .jpg, 100% RGB)
Style folders	27
Named artists (classes.csv)	1,119
classes.csv	80,042 rows x 9 columns
wclasses.csv	81,444 rows x 4 columns
Corrupt / unreadable	0 of 81,444 (full structural scan); 0 truncated in a 1,500-image full-decode sample
Median resolution	2.55 megapixels
Median aspect ratio	0.93 (slightly portrait)

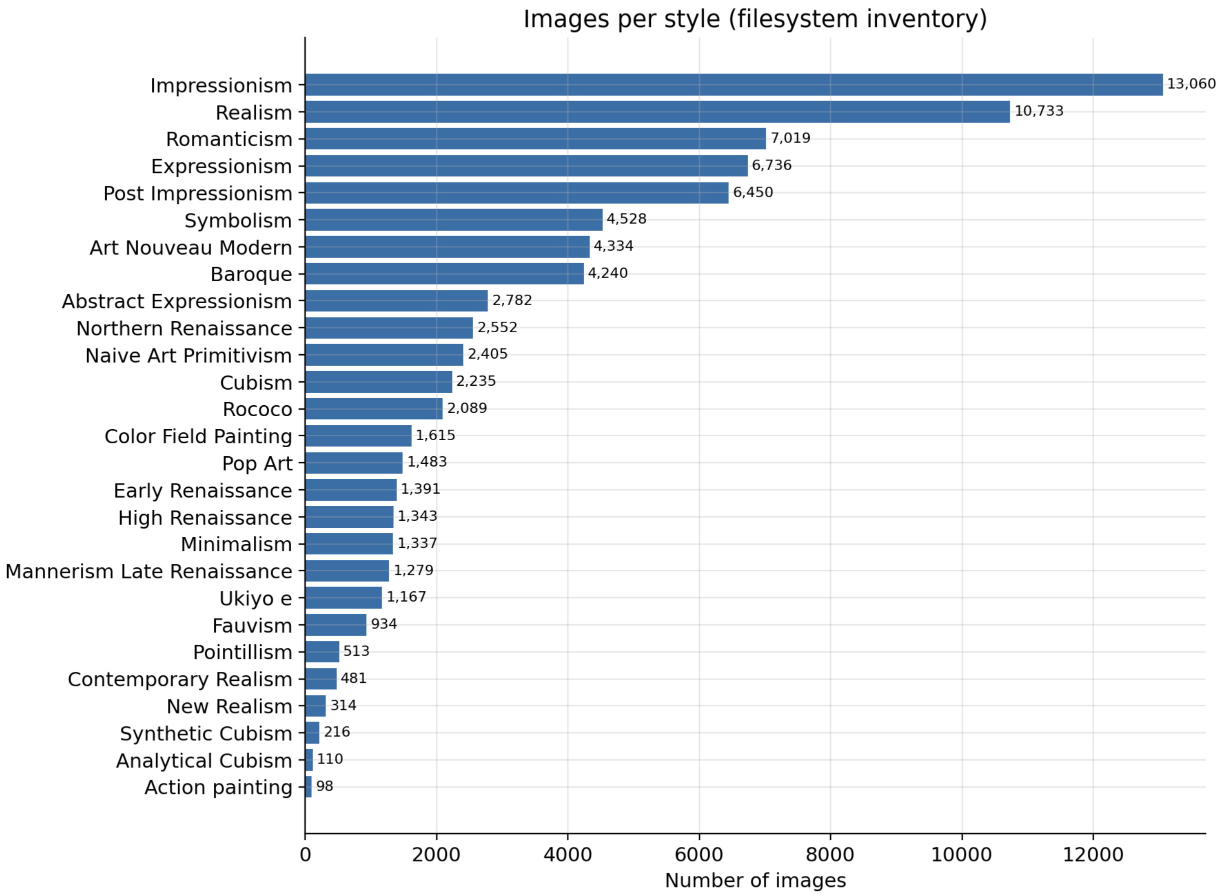

Figure 1. Image count per style (filesystem inventory). The corpus is heavily imbalanced: Impressionism (13,060) and Realism (10,733) dominate, while Action painting (98) and Analytical Cubism (110) are tiny.

4. The two label files, column by column

classes.csv (9 columns): `filename` (Style/artist_title.jpg), `artist` (text, lowercased), `genre` (**actually the STYLE** — a Python-list literal such as `['Impressionism']`; the column is mislabelled at source), `description` (title slug), `phash` (perceptual hash), `width`/`height`, `genre_count` (1–3 style labels), and `subset` (train / test / uncertain artist).

wclasses.csv (4 columns): `file` plus numeric IDs `artist` (0–128), `genre` (129–139) and `style` (140–166) in a shared integer namespace. Two cautions: its `genre` is a *genuine, separate* 11-category concept — not the same thing as `classes.csv`'s mislabelled style — and there is **no legend file**, so the artist and genre IDs cannot be resolved to names.

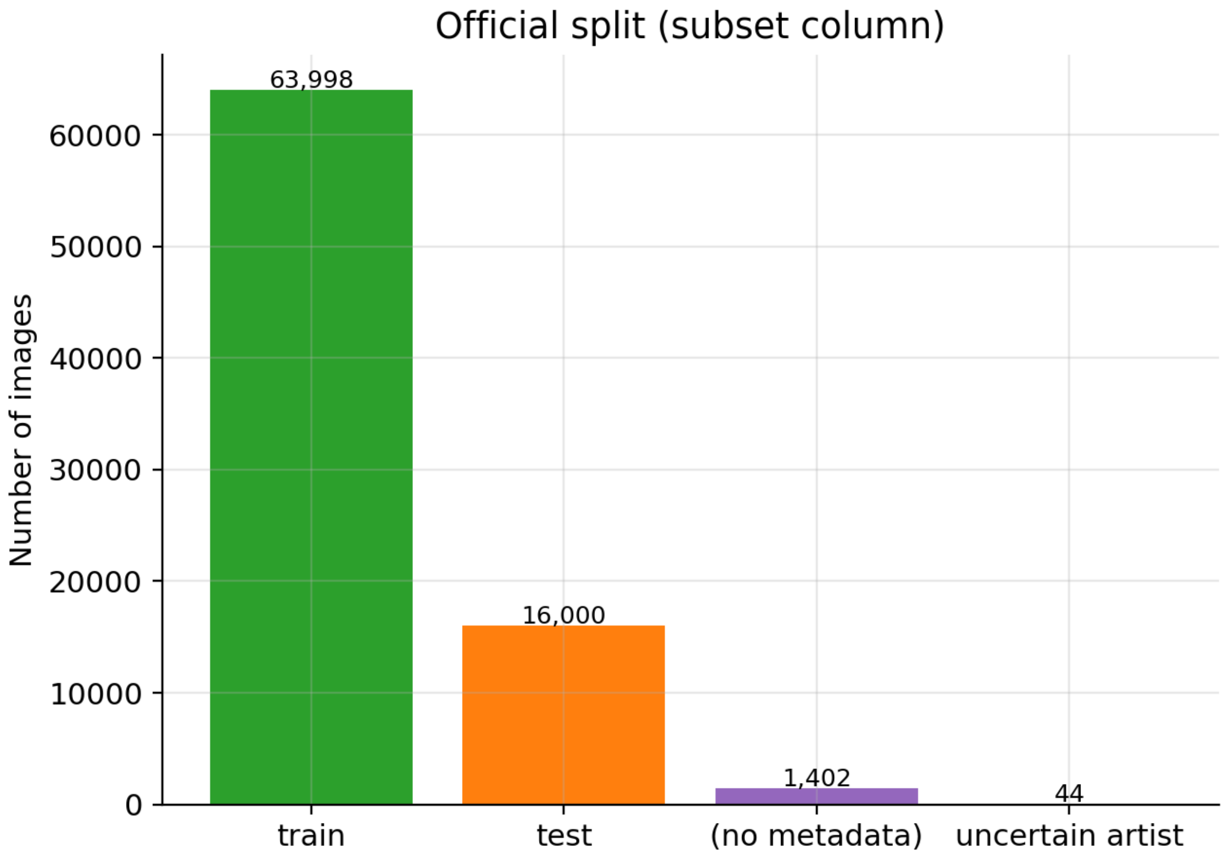

Figure 2. Official split from the subset column: 63,998 train, 16,000 test, 44 'uncertain artist'. 1,402 disk images carry no split label because they are absent from classes.csv.

5. Reconciliation and the encoding defect

Non-ASCII artist names (e.g. Eugène Grasset) are mangled by **two different mojibake encodings** — one on disk, another in `classes.csv` — so a naive exact-path join silently drops every accented filename. The fix is an encoding-robust key that reduces each path to lowercase ASCII alphanumerics, collapsing both corruptions to the same value. Unicode NFKD folding is deliberately avoided because it transliterates the two corrupt byte sequences *inconsistently* and fails to reconcile them. The join runs in **two stages** — an exact-path match first, then the fuzzy key applied only to the residual — so no image that already matched exactly can be reassigned, and the fuzzy step introduces no collisions.

Stage 1 — exact-path match	79,347 images
Stage 2 — encoding-robust fallback	695 accented filenames rescued
Total enriched	80,042 = exactly the 80,042 classes.csv rows
classes.csv rows with no image	0 (every row reconciles to a real image)
Disk images with no classes.csv metadata	1,402 (present only in wclasses.csv)
Disk images missing from wclasses.csv	0 (wclasses is a 1:1 superset of disk)
Residual key collisions	0 — confining the fuzzy match to unmatched images leaves the join unambiguous

6. Data-quality findings

Integrity. A full structural scan of all 81,444 images plus a full-decode pass on a reproducible 1,500-image sample found **0 corrupt or truncated files**; every image is RGB. **Duplicate basenames. 1,349** file names (2,698 files) occur under more than one style folder — the same painting filed under multiple styles; the full path stays unique. **Perceptual duplicates (optional).** Using the provided phash, **22** groups (44 images) are visually identical, 6 of them spanning more than one style.

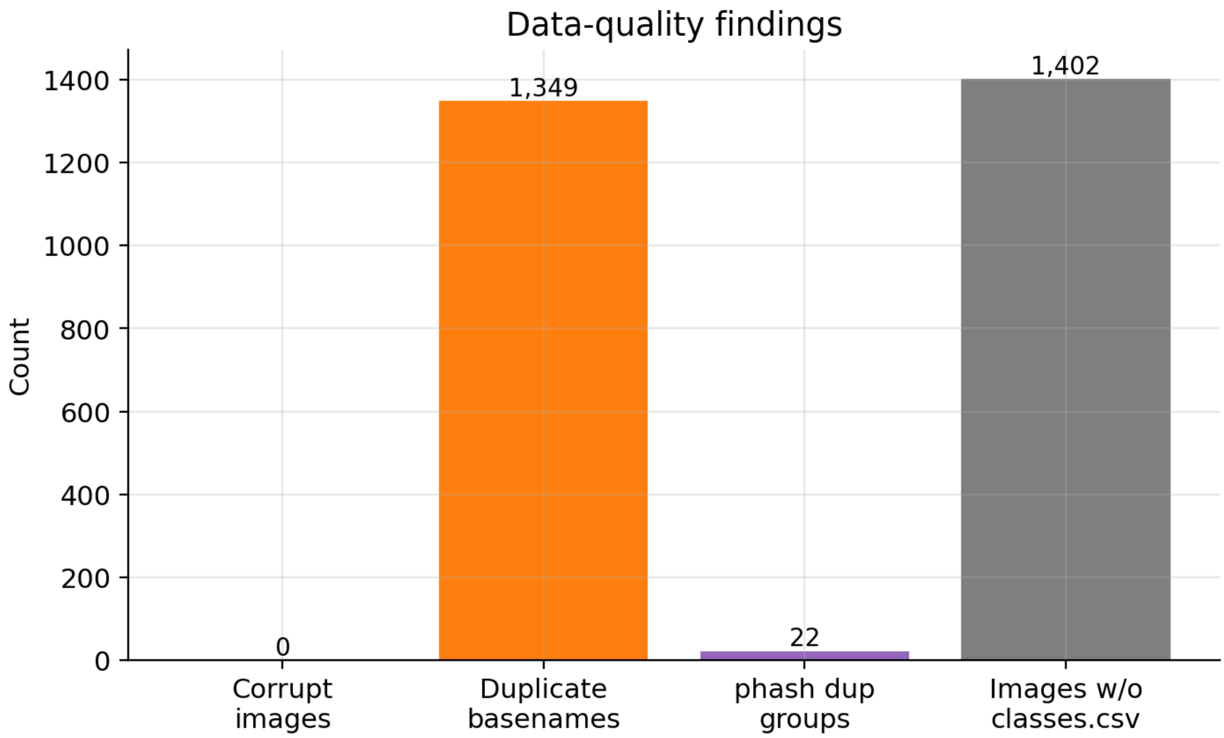

Figure 3. Data-quality summary: zero corrupt images, 1,349 duplicate basenames, 22 perceptual-hash duplicate groups, and 1,402 disk images without classes.csv metadata.

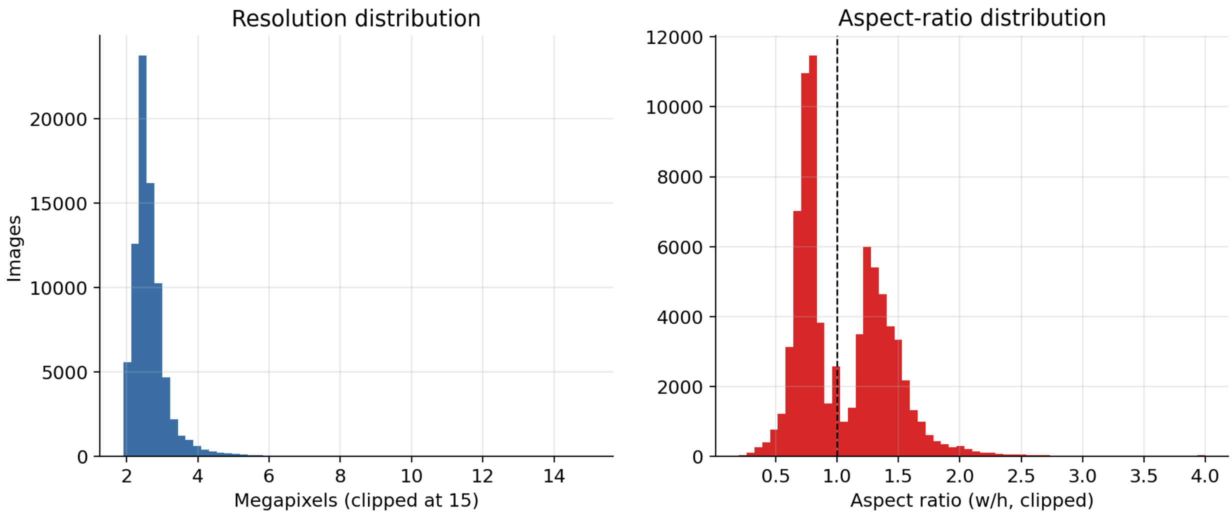

Figure 4. Resolution (median 2.55 MP) and aspect-ratio (median 0.93) distributions. Images are modest-resolution and slightly portrait-oriented.

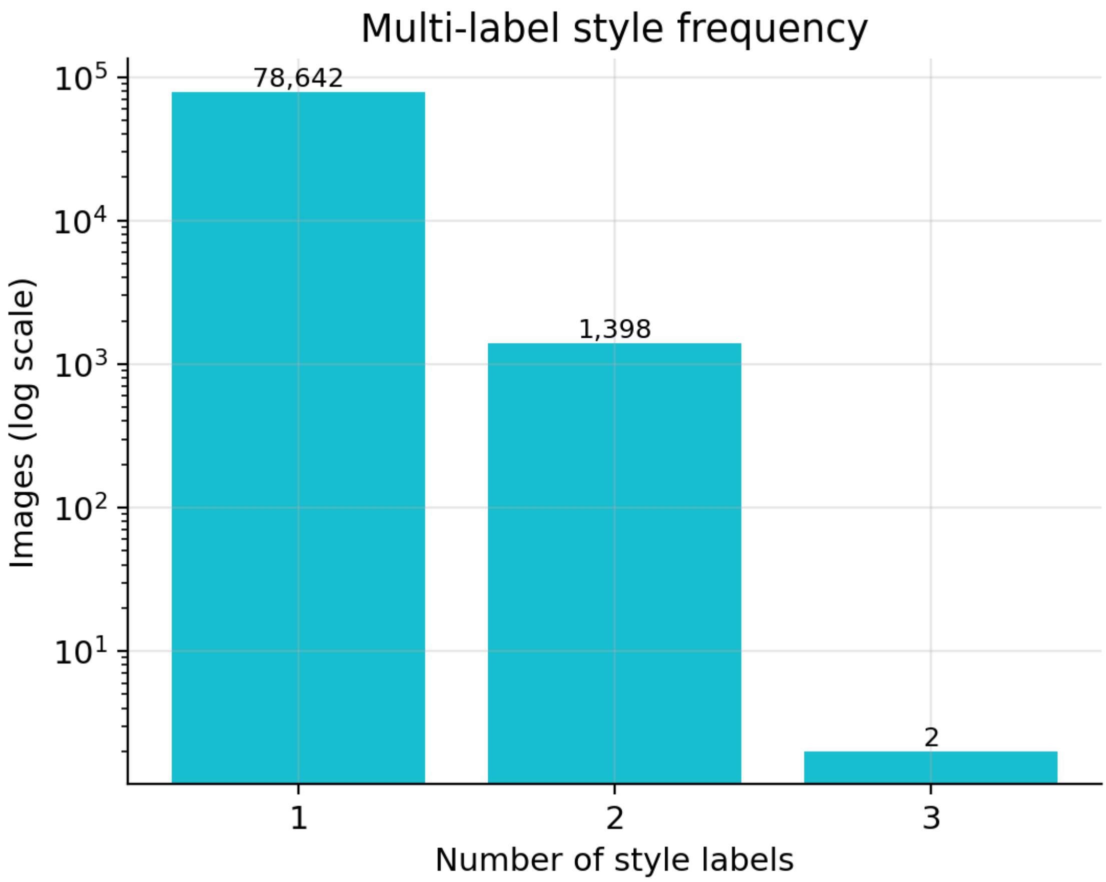

Figure 5. Multi-label style frequency (log scale): most images carry a single style label; ~1,400 carry two and two carry three.

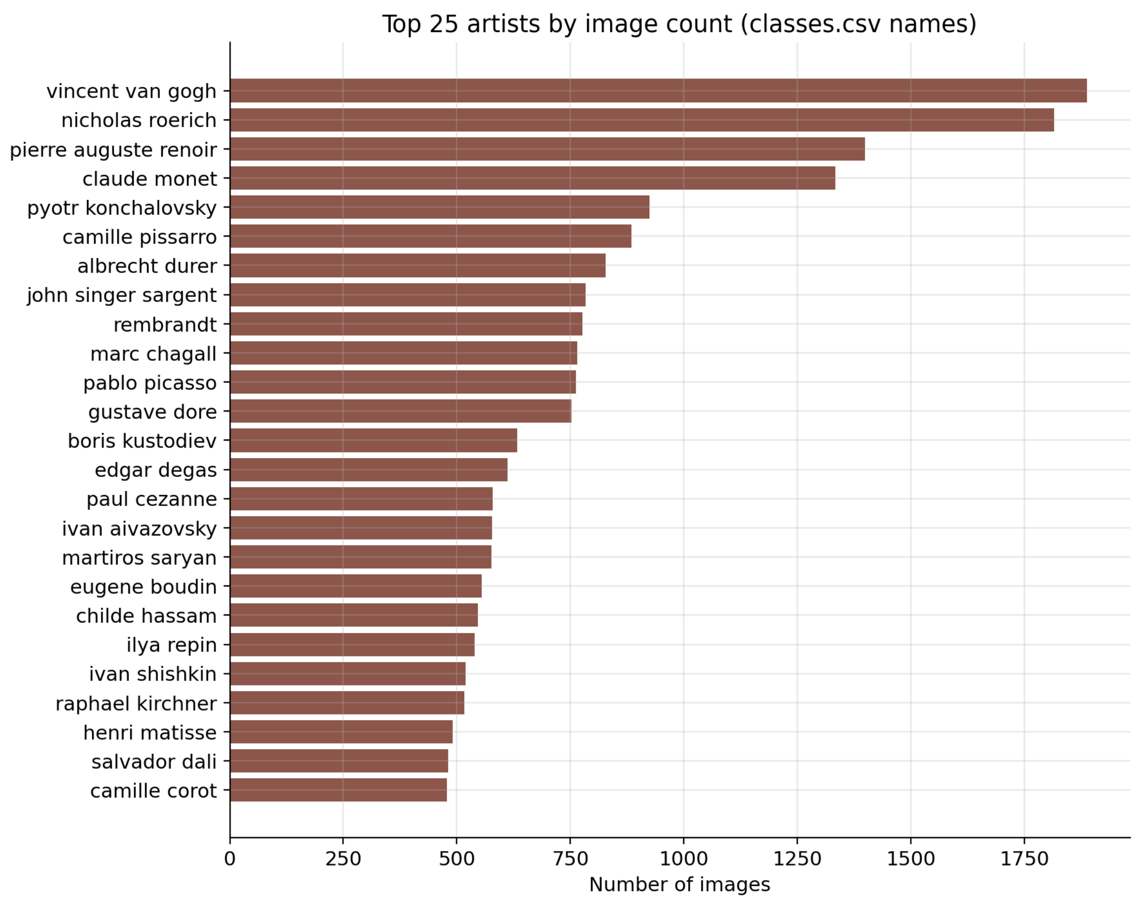

Figure 6. Top 25 artists by image count. A long-tailed distribution over 1,119 named artists.

7. Assumptions & limitations

• No ID legend exists for `wclasses.csv`; its artist and genre IDs are left unresolved rather than guessed.
• The two-stage join is validated, not assumed: the fuzzy fallback is confined to unmatched images and produces **0** residual key collisions, so no image is ever assigned the wrong metadata.
• Width/height come from `classes.csv` metadata rather than re-measured pixels, for efficiency; images without that metadata are excluded from resolution statistics.
• Perceptual-hash duplicate detection covers only images present in `classes.csv`; its 22 groups match the duplicate hashes in `classes.csv` itself, independent of the join.

Full, reproducible detail — including every function, type hint and docstring — is in `notebooks/analysis.ipynb`*, which executes top-to-bottom.*